

# Proyecto JARVIS — Plan de Diseño e Implementación

Asistente de voz local para Windows 11. Se activa por palmadas o wake word, entiende órdenes en español y ejecuta acciones sobre el sistema, tus tareas y Spotify.

**Fecha:** 2026-08-20 **Equipo:** Rafael (Cerebro y Manos) · Hemsy (Oídos, Voz e Interfaz) **Presupuesto:** \$5 USD de créditos OpenAI — única inversión monetaria **Estado:** Diseño aprobado, pendiente de implementación

## 1. Resumen ejecutivo

Jarvis es un proceso Python que corre permanentemente en segundo plano escuchando el micrófono. Detecta dos tipos de activación (una palmada doble, o la frase "oye Jarvis"), transcribe lo que dices, decide qué hacer, lo hace, y te responde hablando.

La decisión arquitectónica central es **local-first**: todo el procesamiento de audio —detección de palmadas, wake word, transcripción y síntesis de voz— corre en tu máquina. La nube solo interviene para una cosa: convertir una frase en español a una llamada de función. Esto tiene tres consecuencias directas:

- 1. **Costo casi nulo.** OpenAI nunca ve audio, solo texto corto. ~\$0.0002 por comando.
- 2. **Latencia baja.** No hay que subir audio ni esperar streaming de vuelta.
- 3. **Privacidad.** Tu micrófono no sale de tu PC salvo la frase ya transcrita.

### Principio de diseño no negociable

**El LLM nunca ejecuta código. Solo elige el nombre de una función de una lista fija y rellena parámetros que después se validan.**

Esto no es paranoia: Jarvis escucha todo lo que suena en la habitación, incluido un video de YouTube o alguien de visita. Si el modelo pudiera generar comandos de shell, cualquier audio en el cuarto sería una superficie de ataque. Por eso: nada de `exec`, nada de `os.system` con strings del modelo, sin herramientas destructivas en el catálogo, y operaciones de archivos restringidas a una lista blanca de carpetas.

## 2. Hardware verificado

Estas specs fueron leídas de la máquina, no estimadas:

| Componente | Valor                       | Implicación                     |
|------------|-----------------------------|---------------------------------|
| CPU        | Intel i7-12650H (10C / 16T) | Sobra para audio en tiempo real |
| RAM        | 15.7 GB                     | Sin problema                    |

| Componente | Valor                                                 | Implicación                                |
|------------|-------------------------------------------------------|--------------------------------------------|
| GPU        | NVIDIA RTX 4050 Laptop, 6 GB VRAM (driver 610.74)     | Whisper en GPU: transcripción en ~300 ms   |
| Python     | 3.11, 3.13 y 3.14 disponibles                         | Usar 3.11 — mejor compatibilidad de wheels |
| Micrófonos | Array Intel Smart Sound (interno) + Redmi Buds 6 Play | Ver advertencia abajo                      |

Advertencia crítica: no usar los Redmi Buds como micrófono

Bluetooth no puede transmitir audio de alta calidad y capturar micrófono al mismo tiempo. Cuando se activa el micrófono de unos auriculares BT, Windows conmuta el perfil de A2DP (estéreo, alta calidad) a **HFP/Hands-Free** (mono, 8-16 kHz). Resultado: cada vez que Jarvis te escuche, la música de Spotify se degrada a calidad de radio de taxi.

**Configuración obligatoria:** entrada = array interno del laptop, salida = lo que quieras. Esto se fija explícitamente en `config.yaml` por índice de dispositivo, nunca "por defecto".

3. Presupuesto real

Lo que cuesta dinero

| Ítem            | Costo           | Obligatorio |
|-----------------|-----------------|-------------|
| Créditos OpenAI | \$5 USD (único) | Sí          |
| TOTAL           | \$5 USD         |             |

Cuánto duran los \$5

Con `gpt-4o-mini` a \$0.15 por millón de tokens de entrada y \$0.60 de salida, y un prompt típico de ~1,200 tokens de entrada (system prompt + definiciones de herramientas) más ~80 de salida:

$$(1200 * 0.15 / 1_000_000) + (80 * 0.60 / 1_000_000) = \sim\$0.00023 \text{ por comando}$$
$$\$5.00 / \$0.00023 = \sim 21,000 \text{ comandos}$$

A 30 comandos diarios entre los dos, eso es **cerca de 2 años**. Y con el router de reglas (\$5.2) los comandos frecuentes ni siquiera tocan la API, así que el número real será bastante mayor. El *prompt caching* de OpenAI abarata todavía más la parte fija del prompt.

**¿Basta gpt-4o-mini?** Sí, holgadamente. La tarea es clasificación de intención con extracción de parámetros — de lo más fácil que hace un LLM. Un modelo de gama alta acá sería desperdiciar dinero sin ganar nada perceptible.

**Verificar antes de cargar los créditos:** el catálogo de OpenAI rota seguido. Revisá la página de pricing por si existe un modelo `mini` más nuevo o más barato. Cualquier modelo de gama "mini" con soporte de *tool calling* sirve — no cambia ni una línea de la arquitectura, solo el string en `config.yaml`.

## Lo que es gratis

`openWakeWord` (Apache-2.0) · `faster-whisper` (MIT) · `Piper TTS` (MIT) · `silero-vad` (MIT) · `sounddevice` (MIT) · `spotipy` (MIT) · `Everything` de voidtools (freeware) · `pycaw` (MIT) · `winsdk` (MIT)

**Costo no monetario:** ~2-3 GB de disco para los modelos de Whisper, wake word y Piper.

## El asterisco: Spotify Free

La Web API de Spotify devuelve **HTTP 403 PREMIUM\_REQUIRED** en todos los endpoints de control de reproducción. No es un bug ni algo que se pueda rodear con código: es una restricción comercial de Spotify. Con cuenta Free hay que entrar por otra puerta.

| Acción                   | Con Premium (API directa)                  | Plan B con Free (el que usaremos)                                                                                |
|--------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Buscar canción           | <code>sp.search()</code>                   | <code>sp.search()</code> — <b>funciona igual en Free</b> , usa Client Credentials y no requiere login de usuario |
| Reproducir un track      | <code>sp.start_playback(uris=[...])</code> | <code>os.startfile("spotify:track:&lt;id&gt;")</code> — la app de escritorio lo abre y lo reproduce              |
| Play / pausa / siguiente | <code>sp.pause_playback()</code>           | <b>SMTC</b> vía <code>winsdk</code> , dirigido a la sesión de Spotify                                            |
| Volumen                  | <code>sp.volume(70)</code>                 | <code>pycaw</code> — volumen del proceso <code>Spotify.exe</code> aisladamente                                   |
| "¿Qué está sonando?"     | <code>sp.current_playback()</code>         | <b>SMTC</b> <code>get_media_properties_async()</code> → título, artista, estado                                  |

**Sobre SMTC:** Windows expone `GlobalSystemMediaTransportControlsSessionManager`, la API que alimenta el panel multimedia del sistema. Permite enumerar sesiones, filtrar la de Spotify por su `AppUserModelId`, y enviarle comandos *dirigidos*. Es mucho más robusto que simular teclas multimedia globales, que se las roba cualquier pestaña del navegador que esté reproduciendo video. Además permite **leer** la canción actual sin tocar la API.

**Limitaciones honestas del Plan B:** hay anuncios entre canciones, y en cuentas Free Spotify a veces ignora el track exacto y arranca un shuffle de artistas relacionados. No hay forma de evitarlo desde el código.

**Mitigación de diseño:** una sola interfaz `SpotifyController` con dos implementaciones (`FreeSpotifyController` y `PremiumSpotifyController`). Si algún día alguno se pone Premium, se cambia una línea de config y el resto del sistema no se entera.

## 4. Stack tecnológico

| Capa                  | Librería                             | Por qué esta y no otra                                                                                                        |
|-----------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Runtime               | <b>Python 3.11</b>                   | 3.13/3.14 aún tienen wheels incompletos para <code>onnxruntime</code> y <code>faster-whisper</code> . 3.11 es el punto dulce. |
| Captura de audio      | <code>sounddevice</code> (PortAudio) | API limpia y nativa de numpy. <code>pyaudio</code> está semiabandonado y compila mal en Windows.                              |
| Detección de palmadas | <b>numpy puro</b>                    | No necesita IA. Una palmada es un transitorio de energía; se detecta con DSP básico. Ver §5.3.                                |

| Capa                 | Librería                                                                      | Por qué esta y no otra                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Wake word            | <code>openWakeWord</code>                                                     | Corre en ONNX sobre CPU en ~4 ms. <b>Trae un modelo pre-entrenado de "hey jarvis"</b> — literalmente lo que queremos. Y permite entrenar modelos propios gratis. |
| VAD (fin de frase)   | <code>silero-vad</code>                                                       | Bastante más preciso que <code>webrtcvad</code> en ambientes ruidosos, y también es ONNX ligero.                                                                 |
| Transcripción        | <code>faster-whisper</code> (CTranslate2)                                     | ~4x más rápido que el Whisper de referencia con la misma precisión. En la 4050 con <code>float16</code> es prácticamente instantáneo.                            |
| Síntesis de voz      | <b>Piper TTS</b> (primaria) + <code>edge-tts</code> (alternativa)             | Piper es 100% local y responde en <100 ms. <code>edge-tts</code> suena mejor pero necesita internet y añade ~500 ms. Ambas detrás de la misma interfaz.          |
| Cerebro              | <b>OpenAI SDK</b> + <code>gpt-4o-mini</code> con <i>tool calling</i>          | Ver §3.                                                                                                                                                          |
| Validación           | <code>pydantic</code>                                                         | Los argumentos que devuelve el LLM se validan contra un esquema antes de tocar nada. Esta es la barrera de seguridad.                                            |
| Búsqueda de archivos | <b>Everything</b> + <code>es.exe</code>                                       | Indexa la MFT de NTFS y responde en milisegundos. Windows Search es lento e inconsistente.                                                                       |
| Tareas / pendientes  | <b>SQLite</b> ( <code>sqlite3</code> , <code>stdlib</code> )                  | Cero dependencias, cero API keys, funciona offline. Sincronizar con Google Tasks es un extra que no necesitamos ahora.                                           |
| Spotify              | <code>spotipy</code> + <code>winsdk</code> + <code>pycaw</code>               | Ver §3.                                                                                                                                                          |
| Config               | <code>pydantic-settings</code> + <code>config.yaml</code> + <code>.env</code> | Secretos en <code>.env</code> (gitignored), ajustes en YAML.                                                                                                     |
| Logs                 | <code>rich</code>                                                             | Consola legible con estados y timings, que es lo que más se depura acá.                                                                                          |
| Interfaz — bandeja   | <code>pystray</code>                                                          | Icono de estado en la bandeja del sistema. Ver §10.                                                                                                              |
| Interfaz — HUD       | <code>pywebview</code> (WebView2)                                             | HUD flotante escrito en HTML/CSS. Fase 4, sujeto a spike. Ver §10.                                                                                               |
| Tests                | <code>pytest</code>                                                           | Ver §9.                                                                                                                                                          |

## requirements.txt inicial

```
sounddevice>=0.4.6
numpy>=1.26
openwakeword>=0.6.0
onnxruntime>=1.17
silero-vad>=5.1
faster-whisper>=1.0
piper-tts>=1.2
edge-tts>=6.1
openai>=1.40
pydantic>=2.7
pydantic-settings>=2.3
spotipy>=2.24
winsdk>=1.0
pycaw>=20240210
pywin32>=306
pyyaml>=6.0
rich>=13.7

# Interfaz (ver §10)
pystay>=0.19          # Fase 2
Pillow>=10.0          # iconos de la bandeja
pywebview>=5.0        # Fase 4, sujeto al spike R9

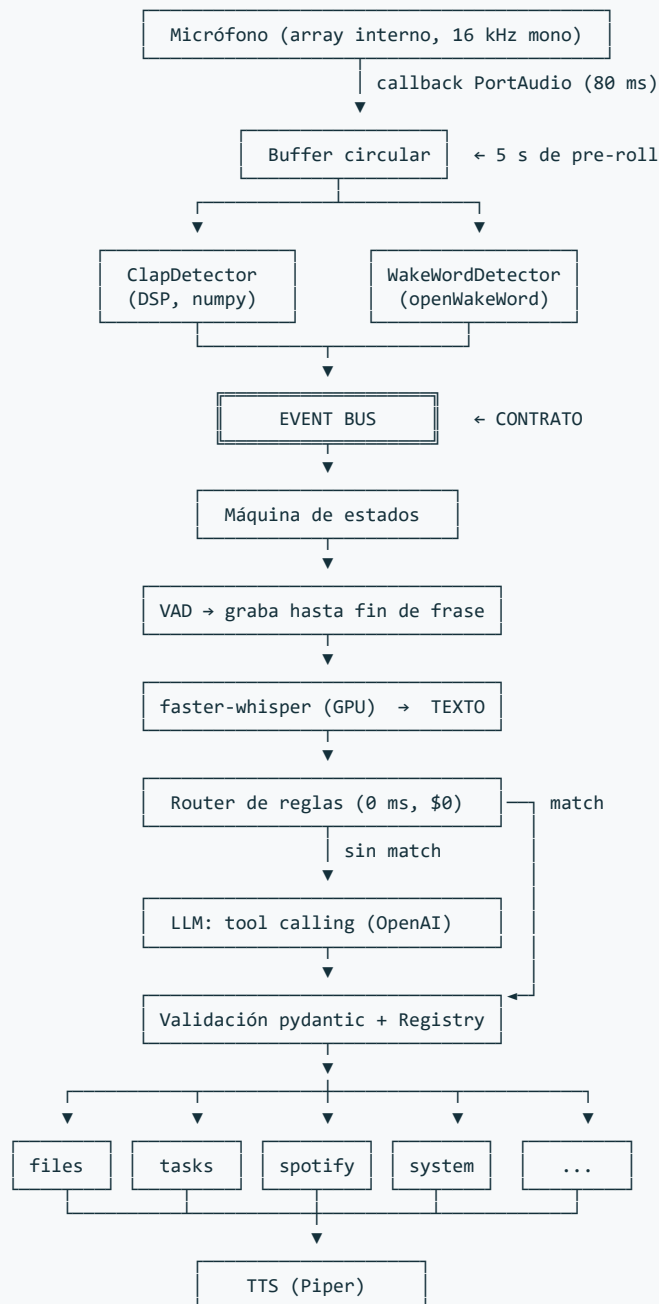
pytest>=8.0
pytest-asyncio>=0.23

# Solo para Whisper en GPU (ver Riesgo R1)
nvidia-cublas-cu12
nvidia-cudnn-cu12
```

---

## 5. Arquitectura

### 5.1 Flujo de datos



### 5.2 El router de reglas (por qué existe)

Antes de gastar una llamada al LLM, un matcher de reglas atiende los ~15 comandos más frecuentes: **pausa**, **siguiente**, **sube el volumen**, **qué hora es**, **qué está sonando**.

Son unas 40 líneas de regex más normalización, y compran tres cosas: latencia cero en lo que más se usa, funcionamiento sin internet para lo básico, y menos gasto de créditos. Lo que no matchea sube al LLM, que es donde está la magia de entender *"ponme algo tranquilo para estudiar"* o *"abrí donde tengo los pdfs de la uni"*.

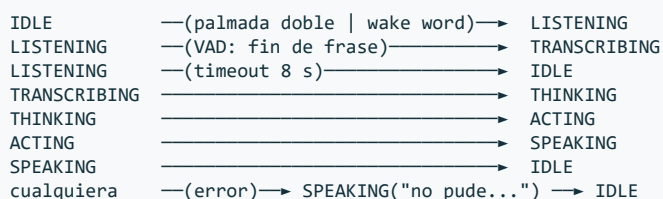
### 5.3 Detección de palmadas (sin IA)

Una palmada tiene una firma acústica muy distinguible: **transitorio de banda ancha, ataque casi vertical y decaimiento muy rápido**. El algoritmo:

1. Energía RMS por ventana de 10 ms.
2. **Umbral adaptativo**: mediana móvil de los últimos 2 s como piso de ruido. Esto es lo que hace que funcione tanto en silencio como con música de fondo — un umbral fijo fallaría siempre en uno de los dos casos.
3. *Onset* = la energía supera el piso por más de N dB.
4. **Filtro de decaimiento**: debe volver cerca del piso en <80 ms. Esto descarta voz, música y risas, que sostienen energía mucho más tiempo.
5. **Filtro de planitud espectral**: una palmada es ruido de banda ancha. Descarta sonidos tonales (un timbre, una nota).
6. **Doble palmada** = dos onsets separados por 150–600 ms.
7. **Periodo refractario** de 800 ms tras un disparo, para no encadenar detecciones.

Se usa doble palmada y no simple a propósito: una palmada suelta se confunde con un portazo o un golpe en la mesa. Dos con el intervalo correcto casi no ocurren por accidente.

### 5.4 Máquina de estados



**El estado **SPEAKING** silencia el detector de wake word.** Sin esto, Jarvis se escucha a sí mismo por los parlantes, se re-dispara y entra en bucle. Es half-duplex: mientras habla, no escucha. Es la solución simple y funciona. La cancelación de eco acústica real (para poder interrumpirlo a media frase) queda como mejora de Fase 4.

## 5.5 Estructura del proyecto

```
jarvis/
├── core/
│   ├── events.py      ← CONTRATO: dataclasses de eventos
│   ├── bus.py         pub/sub asyncio
│   ├── state.py       máquina de estados
│   └── config.py      pydantic-settings
├── ears/              ← HEMSY
│   ├── capture.py     InputStream + buffer circular
│   ├── clap.py        detector de palmadas
│   ├── wakeword.py    openWakeWord
│   ├── vad.py         silero-vad + endpointing
│   └── stt.py         faster-whisper
├── voice/             ← HEMSY
│   ├── base.py        interfaz TTSEngine
│   ├── piper_tts.py
│   └── edge_tts.py
├── brain/             ← RAFAEL
│   ├── router.py      reglas rápidas
│   ├── llm.py         tool calling
│   └── registry.py    registro y despacho de skills
├── skills/            ← RAFAEL
│   ├── base.py        ← CONTRATO: interfaz Skill
│   ├── files.py       Everything / es.exe
│   ├── tasks.py       SQLite
│   ├── spotify.py     spotipy + SMTC + pycaw
│   └── system.py      hora, apps, volumen general
├── ui/                ← HEMSY
│   ├── console.py     salida rich (Fase 1)
│   ├── tray.py        icono de bandeja + estados (Fase 2)
│   └── hud/           HUD flotante (Fase 4)
│       ├── window.py  pywebview
│       └── web/        index.html · style.css · orb.js
├── tests/
│   ├── fixtures/audio/ WAVs de palmadas, ruido, voz
│   └── ...
├── config.yaml
├── .env
└── main.py            (gitignored)
```

## 5.6 Los dos contratos

Estos dos archivos son lo único que ambas personas necesitan acordar. Se escriben **juntos, en la primera sesión, antes que nada más**. Después cada uno trabaja en paralelo sin bloquear al otro.

**core/events.py** — cómo se comunican las capas:

```
@dataclass(frozen=True)
class WakeDetected:
    source: Literal["clap", "wakeword"]
    confidence: float
    timestamp: float

@dataclass(frozen=True)
class SpeechTranscribed:
    text: str
    language: str
    duration_s: float

@dataclass(frozen=True)
class SpeakRequested:
    text: str
    interruptible: bool = True
```

**skills/base.py** — cómo se declara una capacidad:



```
class Skill(Protocol):
    name: str # "spotify_play"
    description: str # lo que lee el LLM para decidir
    params_model: type[BaseModel] # esquema pydantic de los argumentos

    async def execute(self, params: BaseModel) -> SkillResult: ...

@dataclass
class SkillResult:
    ok: bool
    speech: str # lo que Jarvis dice en voz alta
    data: dict | None = None
```

El `Registry` recorre las skills registradas y genera automáticamente el array `tools` que se le manda a OpenAI, derivándolo de `params_model`. Así, **añadir una capacidad nueva es escribir una clase y nada más** — no hay que tocar el prompt ni el orquestador.

## 6. División del trabajo

Como ambos están parejos en Python, la división es simétrica y sigue el eje del pipeline. Ninguno de los dos tracks es "el fácil": Hemsy pelea con DSP y tiempo real; Rafael, con diseño de API e integraciones del SO.

|              | Hemsy — "Oídos, Voz e Interfaz"                                                            | Rafael — "Cerebro y Manos"                                  |
|--------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Dominio      | Todo lo que es señal de audio, más lo que el usuario ve                                    | Todo lo que es decisión y acción                            |
| Carpetas     | ears/ , voice/ , ui/                                                                       | brain/ , skills/                                            |
| Entrega      | Audio del micrófono → texto. Texto → audio del parlante.<br>Estado del sistema → pantalla. | Texto → acción ejecutada + frase de respuesta.              |
| Se pelea con | PortAudio, buffers, umbrales adaptativos, VRAM, latencia, transparencia de ventanas        | Esquemas de tool calling, APIs de Windows, OAuth de Spotify |
| Módulos      | capture · clap · wakeword · vad · stt · piper_tts · edge_tts · tray · hud                  | router · llm · registry · files · tasks · spotify · system  |

**Compartido (se hace en pareja):** `core/events.py`, `core/bus.py`, `core/state.py`, `core/config.py` y `main.py`. Son pocos archivos y son las costuras del sistema — vale la pena escribirlos juntos.

### Cómo evitar bloquearse mutuamente

Después de la Fase 0, cada uno programa contra un doble del otro:

- **Hemsy** publica eventos reales en el bus, y un consumidor de prueba los imprime en consola. No necesita que exista ninguna skill.
- **Rafael** usa `main.py --text-mode`, un flag que **salta todo el pipeline de audio** y lee comandos por teclado desde stdin. No necesita micrófono ni GPU.

Ese flag `--text-mode` es la pieza más importante para la productividad del equipo: permite desarrollar y depurar el cerebro entero sin hablarle a la computadora, y hace que los tests end-to-end sean triviales. **Vale la pena construirlo en la Fase 0.**

## 7. Estrategia de trabajo en equipo

### 7.1 La ventaja que ya está en el diseño

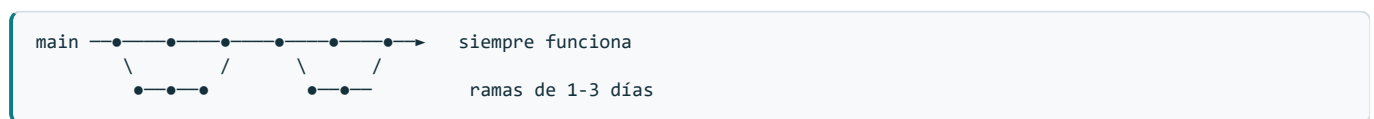
La división de \$6 no es solo un reparto de tareas: es **una estrategia de merge**. Rafael y Hemsy tocan carpetas disjuntas, así que la inmensa mayoría de los commits de uno ni siquiera rozan archivos del otro. Git no tiene nada que resolver.

| Zona                          | Dueño             | Riesgo de conflicto                               |
|-------------------------------|-------------------|---------------------------------------------------|
| brain/ , skills/              | Rafael            | Ninguno — Hemsy no entra                          |
| ears/ , voice/ , ui/          | Hemsy             | Ninguno — Rafael no entra                         |
| core/ , main.py , config.yaml | <b>Compartido</b> | <b>Aquí se concentra el 100% del riesgo</b>       |
| requirements.txt              | Compartido        | Bajo, pero conflictúa fácil por líneas adyacentes |

Todo lo que sigue existe para proteger esa cuarta fila. El resto se cuida solo.

### 7.2 Modelo de ramas: *trunk-based* con ramas cortas

Nada de GitFlow, ni ramas `develop` , `release` o `hotfix` . Para dos personas eso es ceremonia sin beneficio. El modelo es:



#### Reglas:

1. **main siempre funciona.** Si alguien clona y ejecuta, arranca. No se rompe `main` "un ratito"; en un equipo de dos, `main` roto bloquea al 50% de la plantilla.
2. **Nunca se commitea directo a `main`.** Todo entra por Pull Request.
3. **Ramas cortas: de 1 a 3 días.** Una rama de dos semanas es una bomba de conflictos. Si una tarea es más larga, se parte en piezas que se puedan fusionar por separado.
4. **Una rama = una cosa.** No se mezcla "wake word" con "arreglar el TTS".

**Nombres de rama:** `<nombre>/<área>-<qué>`

```
rafael/brain-tool-calling
rafael/skills-spotify-smtc
hemsy/ears-wakeword
hemsy/ui-tray-icon
```

El prefijo con el nombre hace que `git branch -a` se lea de un vistazo y deja claro a quién preguntarle por una rama abandonada.

## 7.3 El ciclo diario

```
# 1. Antes de empezar a trabajar – siempre, todos los días
git checkout main
git pull origin main

# 2. Rama nueva desde main actualizado
git checkout -b hemsy/ears-vad

# 3. Trabajar. Commits pequeños y frecuentes (no hace falta que compilen)
git add -A
git commit -m "feat(ears): endpointing con silero-vad"

# 4. Si la rama lleva más de un día, traer los cambios de main
git pull --rebase origin main

# 5. Subir y abrir PR
git push -u origin hemsy/ears-vad
gh pr create --fill
```

**Sobre el paso 4:** usar `--rebase` y no `merge`. Mantiene el historial lineal y evita llenar `main` de commits "Merge branch 'main' into...". Regla simple: **rebase en tu rama, squash al entrar a `main`, nunca rebase de algo ya publicado que el otro esté usando.**

## 7.4 Pull Requests: por qué, siendo solo dos

La tentación de saltarse los PR con dos personas es fuerte y es un error. La razón no es control de calidad — es que **cada uno es dueño exclusivo de la mitad del código**. Sin revisión cruzada, el *bus factor* del proyecto es 1 en cada mitad: si Hemsy desaparece una semana, Rafael no sabe cómo funciona el detector de palmadas, y viceversa.

El PR es el único momento en que cada uno mira el código del otro. Esa es su función real.

| Quién revisa                                                                |                             |
|-----------------------------------------------------------------------------|-----------------------------|
| PR de Rafael ( <code>brain/</code> , <code>skills/</code> )                 | Hemsy                       |
| PR de Hemsy ( <code>ears/</code> , <code>voice/</code> , <code>ui/</code> ) | Rafael                      |
| PR que toca <code>core/</code>                                              | <b>Los dos, obligatorio</b> |

**Qué mirar al revisar.** No estilo ni nombres de variables — eso es ruido. La pregunta única es: *si mañana tengo que tocar esto sin que estés, ¿lo entiendo?* Si la respuesta es no, el comentario correcto no es "cambiá esto", es "explicame esto" — y a veces la respuesta es un comentario en el código, no un refactor.

**Merge con squash.** Cada PR entra a `main` como **un solo commit** con el título del PR. Los 12 commits de "wip", "arreglo typo", "ahora sí" quedan en el historial de la rama y no ensucian `main`. Con squash, `git log main --oneline` se lee como la lista de features del proyecto.

**Autofusión:** el que abre el PR lo fusiona después de la aprobación. El revisor aprueba, no fusiona — así el autor controla el momento.

## 7.5 La regla que evita el 90% de los dolores: contratos primero

El único archivo que puede hacerles perder una tarde es `core/events.py`. Si Rafael añade un campo a `SpeechTranscribed` mientras Hemsy está reescribiendo quién lo emite, el merge duele y, peor, el código queda roto de formas silenciosas.

**Los cambios a `core/` van en su propio PR, pequeño, primero, y avisando.** Nunca mezclados dentro de un PR de feature.

El flujo cuando alguien necesita un evento o campo nuevo:

1. Avisar por el canal que usen: "necesito un campo `confidence` en `SpeechTranscribed`".
2. PR mínimo que solo toca `core/events.py`. Se revisa en minutos.
3. Se fusiona a `main`.
4. **Los dos hacen `git pull`**.
5. Recién entonces cada uno construye encima.

Cuesta diez minutos y evita la clase de conflicto que no la resuelve Git, sino una llamada.

## 7.6 Cadencia

| Cuándo                      | Qué                                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <b>Cada mañana</b>          | <code>git pull origin main</code> antes de tocar nada. No negociable.                     |
| <b>Al abrir un PR</b>       | Avisar. Un PR sin revisar 24 h bloquea a quien lo abrió.                                  |
| <b>Al empezar cada fase</b> | Sync de 20 min: qué construye cada uno, qué contratos cambian, qué necesita uno del otro. |
| <b>Al cerrar cada fase</b>  | Demo funcionando + <code>git tag fase-N</code> . Es el punto de retorno seguro.           |

Las fases de §8 ya están diseñadas para que ambos tengan trabajo en paralelo en todas. Ninguno queda esperando — ese fue el criterio para ordenarlas así.

## 7.7 Organización de tareas

**GitHub Issues + Milestones.** Un milestone por fase ( `Fase 1 – Escucha y responde` ), un issue por tarea, etiquetas `ears` / `brain` / `skills` / `ui` / `core`. Cada PR cierra su issue con `Closes #12` en la descripción.

Es suficiente para dos personas y vive junto al código. Un tablero externo (Trello, Notion) añade un sitio más que sincronizar a mano y se desactualiza en dos semanas.

## 7.8 Configuración del repositorio

Conviene dejarlo puesto desde el principio, cuando cuesta un minuto:

- **Proteger `main`**: exigir Pull Request antes de fusionar, y al menos 1 aprobación. Es gratis en repos públicos. Convierte las reglas de §7.2 en algo que el servidor hace cumplir, en vez de algo que hay que recordar.
- **Squash merge como única opción**: desactivar `merge commit` y `rebase merge` en los ajustes del repo. Así nadie se equivoca de botón.
- **Borrado automático de ramas** al fusionar, para que la lista no se llene de basura.
- `.github/CODEOWNERS`: asigna el revisor automáticamente según la carpeta tocada. Ya está en el repo.

## 7.9 Chuleta de emergencia

```
# Me equivoqué de rama y trabajé sobre main sin commitear
git stash && git checkout -b rafael/lo-que-sea && git stash pop

# Committeé en main por error (aún sin push)
git branch rafael/rescate && git reset --hard origin/main && git checkout rafael/rescate

# Conflicto al hacer rebase
git status                # ver qué archivos
# ...editar y resolver...
git add <archivo> && git rebase --continue
git rebase --abort         # si se complica, abortar y pedir ayuda

# Quiero ver qué cambió el otro desde ayer
git fetch origin && git log --oneline main..origin/main

# Subí algo secreto por error --> NO intentar arreglarlo solo:
# 1. Revocar la credencial inmediatamente (OpenAI / Spotify)
# 2. Generar una nueva
# 3. Después, limpiar el historial
```

Ese último caso es el único de la lista que es una emergencia real. **Borrar el commit no sirve de nada: la clave ya se filtró en cuanto se subió.** Lo primero es siempre revocar, no limpiar. El repositorio es público, así que el margen es de minutos, no de horas.

## 8. Fases

Cada fase termina en algo que funciona y se puede demostrar. No hay fase que entregue "la mitad de un pipeline".

### Fase 0 — Contratos y esqueleto (los dos juntos)

Solo esto: `events.py`, `skills/base.py`, `config.py`, el bus, y `main.py --text-mode` con una skill de juguete ( `ping` → responde "pong" por consola).

**Terminada cuando:** `python main.py --text-mode`, escribes `ping`, imprime `pong`. Es poco código, pero es lo que desbloquea todo el paralelismo posterior.

### Fase 1 — "Escucha y responde" (primer corte vertical)

| Hemsy                                                                 | Rafael                                                                 |
|-----------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>capture.py</code> + selección explícita de dispositivo          | <code>router.py</code> con 5 reglas                                    |
| <code>wakeword.py</code> con el modelo "hey jarvis"                   | <code>skills/system.py</code> : hora, fecha, saludo                    |
| <code>stt.py</code> con faster-whisper en GPU                         | Conectar <code>SkillResult.speech</code> → <code>SpeakRequested</code> |
| <code>piper_tts.py</code>                                             |                                                                        |
| <code>ui/console.py</code> — estados y latencias en <code>rich</code> |                                                                        |

**Terminada cuando:** decís "oye Jarvis, qué hora es" y responde en voz alta. **Todavía sin OpenAI.** Este hito ya es un producto usable, y es donde se descubren el 90% de los problemas reales (latencia, eco, ruido, dispositivos).

## Fase 2 — El cerebro y las palmadas

| Hemsy                                                               | Rafael                                                                 |
|---------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>clap.py</code> + dataset de fixtures de audio                 | <code>llm.py</code> : tool calling contra OpenAI                       |
| <code>vad.py</code> + endpointing (fin de frase real)               | <code>registry.py</code> : generar <code>tools</code> desde las skills |
| Gate half-duplex durante <code>SPEAKING</code>                      | Manejo de errores y fallbacks del LLM                                  |
| <code>ui/tray.py</code> — <b>icono de bandeja con estados</b> (§10) |                                                                        |

**Terminada cuando:** doble palmada activa, una orden que ninguna regla cubre ("*decime algo que me anime*") llega al LLM y se resuelve, y **el icono de bandeja refleja el estado sin necesidad de mirar la terminal**. Desde este punto Jarvis se puede usar a diario, no solo demostrar.

## Fase 3 — Las manos

Todo de Rafael, con Hemsy dando soporte en integración:

- `skills/files.py` — Everything/ `es.exe`, con lista blanca de carpetas
- `skills/tasks.py` — SQLite: agregar, listar, completar pendientes
- `skills/spotify.py` — `SpotifyController` + implementación Free (search, startfile, SMTC, pycaw)

Hemsy en paralelo:

- Afinar umbrales con datos reales de uso.
- Probar `medium` / `large-v3-turbo` de Whisper para ver si mejora la transcripción de nombres propios y títulos de canciones sin perder latencia.
- **Spike de transparencia de `pywebview`** (~30 min, riesgo R9). Decide si el HUD de la Fase 4 va en WebView2 o cae a Tkinter. Hacerlo acá y no en Fase 4 evita descubrir el problema cuando ya hay HTML escrito.

**Terminada cuando:** los tres verbos del pedido original funcionan — "*ubicá la carpeta X*", "*qué tengo pendiente*", "*poné tal canción*" — y el spike del HUD tiene veredicto.

## Fase 4 — HUD y pulido (cuando lo anterior esté sólido)

**Hemsy — el HUD** (§10): ventana flotante translúcida con orbe reactivo al nivel del micrófono, transcripción en vivo y respuesta. Antes de escribir el HTML, invocar la skill `ui-ux-pro-max` para fijar paleta, escala tipográfica, especificación de estados y motion. Recién en este punto tiene sentido: ya se sabe qué estados emite el bus de verdad.

**Rafael — profundidad del cerebro:** historial de conversación para preguntas de seguimiento ("*y la siguiente?*"), más skills (clima, notas, WhatsApp).

**Compartido:** *barge-in* con cancelación de eco · arranque automático con Windows.

# 9. Estrategia de testing

El problema obvio de un proyecto de voz es que parece que hay que hablarle a la computadora para probarlo. No es así, y evitarlo es lo que hace el desarrollo llevadero.

| Capa                                              | Cómo se testea                                                                                                                               | Sin necesidad de   |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <code>clap.py</code>                              | Fixtures WAV: 30 palmadas + 30 no-palmadas (portazos, teclado, risas, música). El test corre el detector y exige precision y recall mínimos. | Micrófono          |
| <code>wakeword.py</code> ,<br><code>vad.py</code> | Igual, con WAVs grabados una sola vez.                                                                                                       | Micrófono          |
| <code>stt.py</code>                               | WAVs conocidos, comparar con transcripción esperada.                                                                                         | Micrófono          |
| <code>skills/*</code>                             | Llamar <code>execute()</code> directo con params contruidos a mano.                                                                          | Voz, LLM           |
| <code>router.py</code>                            | Tabla de frases → skill esperada. Rapidísimo.                                                                                                | Voz, LLM, red      |
| <code>llm.py</code>                               | Ciente de OpenAI mockeado. Verificar que <i>"pon música triste"</i> produce <code>spotify_play(query=...)</code> .                           | Voz, red, créditos |
| End-to-end                                        | <code>main.py --text-mode</code> con un script de comandos.                                                                                  | Micrófono          |

Lo primero que hay que hacer en la Fase 2 es grabar el dataset de fixtures. Media hora de grabar palmadas y ruidos convierte el ajuste del detector de "probar a ver qué pasa" en un ciclo medible de segundos.

## 10. Interfaz de usuario

Jarvis se maneja por voz, así que la interfaz no es el canal principal — es **retroalimentación de estado**. Su trabajo es responder una sola pregunta: *¿me está escuchando ahora mismo?* Sin eso le hablás a la nada y no sabés si te oyó.

Por eso la interfaz no es una decisión binaria sino una escalera de tres peldaños, y **el bus de eventos (§5.1) hace que subirlos sea gratis**: cualquier UI es un suscriptor más que escucha `WakeDetected` , `SpeechTranscribed` y los cambios de estado, y pinta. Añadir interfaz no obliga a refactorizar nada.

### Peldaño 1 — Consola `rich` · Fase 1

Estado, transcripción y latencias en la terminal. Costo cero. Sirve para desarrollar y depurar; no sirve para usarlo a diario porque exige tener la terminal a la vista.

### Peldaño 2 — Icono de bandeja · Fase 2 · obligatorio

`pystray` más notificaciones nativas de Windows. El icono cambia de color según el estado:

| Estado                                      | Color           |
|---------------------------------------------|-----------------|
| <code>IDLE</code>                           | Gris            |
| <code>LISTENING</code>                      | Azul (pulsante) |
| <code>THINKING</code> / <code>ACTING</code> | Ámbar           |
| <code>SPEAKING</code>                       | Verde           |
| Error                                       | Rojo            |

Menú contextual con: pausar la escucha, recargar config, salir. Son ~150 líneas y es lo que convierte el proyecto de "script que corro en una terminal" a "algo que uso". **No es opcional en la práctica** — sin retroalimentación de estado, la experiencia se cae.

### Peldaño 3 — HUD flotante · Fase 4

Ventana sin bordes, siempre encima, fondo transparente, arrastrable. Contiene un orbe que reacciona al nivel del micrófono en tiempo real, la frase transcrita mientras se procesa, y la respuesta de Jarvis. Aparece al activarse y se desvanece unos segundos después de responder — no vive permanentemente en pantalla.

**Tecnología elegida:** `pywebview` + **HTML/CSS/canvas**.

| Opción                                          | Veredicto                                                                                                                                                              |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pywebview</code> + <b>HTML/CSS/canvas</b> | <b>Elegida.</b> El orbe con glow, la onda de audio y las transiciones son casi triviales en CSS y canvas, y dolorosas en Qt. Usa WebView2, ya instalado en Windows 11. |
| <code>PySide6</code> / <code>PyQt6</code>       | Nativo y potente, pero muy verboso; cada animación decente cuesta trabajo real.                                                                                        |
| <code>Tkinter</code>                            | <b>Fallback probado.</b> Su opción <code>-transparentcolor</code> de Windows funciona con seguridad. Se ve bastante peor, pero no tiene dependencias.                  |

**Spike obligatorio antes de comprometerse (Fase 3, ~30 min):** la transparencia real de ventana sobre WebView2 tiene fama de ser inconsistente entre versiones. Hay que verificar que `transparent=True` + `frameless=True` + `on_top=True` se comportan en esta máquina **antes** de escribir el HUD. Si no se porta, se cae a Tkinter y se ajustan las expectativas visuales. El resultado del spike se anota acá mismo.

**Diseño visual:** cuando se arranque el HUD, usar la skill `ui-ux-pro-max` para definir paleta, escala tipográfica, especificación de estados y motion. Invocarla en ese momento y no antes: necesita saber qué estados reales emite el bus para que la especificación sirva de algo. Dirección estética de partida: oscuro, translúcido, acento cian, glow suave, tipografía condensada — el registro visual de Iron Man sin caer en la parodia.

### Lo que queda fuera

**Una web UI completa** (FastAPI + WebSocket + React, con panel de historial, gestión de tareas y configuración) queda descartada. Es un proyecto en sí mismo y no aporta nada a un asistente que se maneja por voz. Si algún día quieren un panel de administración, es un segundo producto, no parte de este.

### A quién le toca

**Los peldaños 2 y 3 son de Hemsy.** Encaja por dos razones: su carga baja en Fase 3 —para entonces solo está afinando umbrales— mientras Rafael todavía tiene tres skills por construir; y A ya tiene en la mano el dato que el orbe necesita para animarse, que es el nivel de audio en tiempo real del buffer de captura. Nadie más lo tiene tan a mano.



## 11. Riesgos y mitigaciones

| #  | Riesgo                                                                                                                                                               | Probabilidad         | Mitigación                                                                                                                                                                                                                                                                                                 |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | <b>cuDNN/cuBLAS:</b> <code>faster-whisper</code> en GPU falla en Windows por DLLs faltantes. Es el problema clásico de este stack.                                   | Alta                 | <code>pip install nvidia-cublas-cu12 nvidia-cudnn-cu12</code> y añadir sus <code>lib/</code> con <code>os.add_dll_directory()</code> al arrancar. Fallback automático a <code>device="cpu", compute_type="int8"</code> , que igual funciona (más lento). <b>Probar esto en la Fase 1, no al final.</b>     |
| R2 | <b>El wake word "hey jarvis" no reconoce acento español.</b> El modelo pre-entrenado se entrenó con voces en inglés.                                                 | Media                | Escalera de mitigación: (a) probarlo el primer día de la Fase 1; (b) si falla, <code>openWakeWord</code> trae un notebook de entrenamiento gratuito que genera miles de muestras sintéticas con Piper — se puede entrenar <b>"oye Jarvis"</b> en español; (c) último recurso: activación solo por palmada. |
| R3 | <b>Falsos positivos de palmadas</b> con portazos, teclado mecánico, aplausos en un video.                                                                            | Media                | Doble palmada + filtro de decaimiento + planitud espectral + periodo refractario. Se mide contra el dataset de fixtures, no a ojo.                                                                                                                                                                         |
| R4 | <b>Eco del TTS</b> re-dispara el wake word y hace bucle.                                                                                                             | Alta si no se maneja | Gate half-duplex en el estado <code>SPEAKING</code> (§5.4). Es una línea de lógica y resuelve el caso por completo en v1.                                                                                                                                                                                  |
| R5 | <b>Spotify Free</b> ignora el track y hace shuffle, más los anuncios.                                                                                                | Alta                 | Aceptado y documentado. Interfaz <code>SpotifyController</code> deja la puerta abierta a Premium sin refactor.                                                                                                                                                                                             |
| R6 | <b>Teclas multimedia</b> capturadas por el navegador en lugar de Spotify.                                                                                            | Media                | Por eso usamos SMTC dirigido a la sesión de Spotify por <code>AppUserModelId</code> , y no <code>keybd_event</code> global.                                                                                                                                                                                |
| R7 | El LLM alucina una skill que no existe o manda params inválidos.                                                                                                     | Media                | Validación pydantic antes de ejecutar; si falla, se devuelve el error al modelo para un reintento, con máximo 2 vueltas y luego una disculpa hablada.                                                                                                                                                      |
| R8 | <b>Comandos accidentales desde audio ambiente</b> (un video, una visita).                                                                                            | Media                | Catálogo de herramientas sin operaciones destructivas. Lista blanca de carpetas para <code>files</code> . Nada de shell arbitrario. Ver §1.                                                                                                                                                                |
| R9 | <b>Transparencia de ventana en <code>pywebview</code> / <code>WebView2</code></b> inconsistente entre versiones — el HUD sale con fondo negro en vez de translúcido. | Media                | Spike de ~30 min en <b>Fase 3</b> , antes de escribir una línea de HTML. Si falla, fallback a Tkinter con <code>-transparentcolor</code> , que sí funciona con seguridad en Windows, aceptando un resultado visual más pobre. El HUD es Fase 4 y no bloquea nada anterior.                                 |

## 12. Checklist de arranque

Antes de escribir la primera línea:

- ☐ Crear el repo e invitar a la otra persona
- ☐ `py -3.11 -m venv .venv` (Python 3.11, no 3.13 ni 3.14)
- ☐ Instalar [Everything](#) y habilitar la CLI `es.exe` en el PATH
- ☐ Crear la app en el [Spotify Developer Dashboard](#) → Client ID y Secret
- ☐ Cargar los \$5 en la cuenta de OpenAI y generar la API key
- ☐ `.env` con `OPENAI_API_KEY`, `SPOTIFY_CLIENT_ID`, `SPOTIFY_CLIENT_SECRET` — y `.env` en `.gitignore` desde el primer commit

- [ ] Identificar el índice del array de micrófono interno con `python -m sounddevice` y fijarlo en `config.yaml`
- [ ] `openwakeword.utils.download_models()` para bajar los modelos pre-entrenados
- [ ] Descargar una voz de Piper en español (`es_MX-claude-high` o `es_ES-davefx-medium`)

## 13. Decisiones tomadas y descartadas

| Decisión                   | Elegido                                  | Descartado                            | Razón                                                                                                                                   |
|----------------------------|------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Dónde vive la inteligencia | Local-first, LLM solo como enrutador     | OpenAI Realtime API (voz a voz)       | Realtime cobra por token de <b>audio</b> , ~2 órdenes de magnitud más caro. Los \$5 darían para ~40-60 minutos totales de conversación. |
| Entendimiento de intención | LLM + router de reglas por delante       | Solo reglas (regex/fuzzy)             | Sin LLM no entiende <i>"ponme algo tranquilo para estudiar"</i> . Deja de sentirse como Jarvis y se siente como un menú telefónico.     |
| Transcripción              | <code>faster-whisper</code> local en GPU | Whisper API de OpenAI                 | La 4050 lo hace gratis y más rápido que subir el audio. La API costaría \$0.006/min.                                                    |
| Activación                 | Doble palmada + wake word                | Palmada simple                        | Una palmada suelta se confunde con un portazo.                                                                                          |
| Control de Spotify         | SMTC dirigido                            | Teclas multimedia globales            | Las globales se las roba el navegador si está reproduciendo video.                                                                      |
| Almacenamiento de tareas   | SQLite local                             | Google Tasks / Todoist API            | YAGNI. Cero setup, cero auth, funciona offline. Se puede sincronizar después.                                                           |
| Búsqueda de archivos       | Everything + <code>es.exe</code>         | Windows Search / <code>os.walk</code> | Everything responde en milisegundos indexando la MFT.                                                                                   |
| Duplex del audio           | Half-duplex (no escucha mientras habla)  | Cancelación de eco acústica           | AEC real es un proyecto en sí mismo. El gate resuelve el 100% del problema en v1.                                                       |
| Interfaz                   | Escalera: consola → bandeja → HUD        | Elegir una sola de entrada            | El bus de eventos hace que cada peldaño sea aditivo y sin refactor. No hay que decidirlo hoy.                                           |
| Tecnología del HUD         | <code>pywebview</code> + HTML/CSS/canvas | PySide6 / Qt · Tkinter                | El orbe con glow y la onda de audio son triviales en CSS y dolorosas en Qt. Tkinter queda como fallback probado si el spike R9 falla.   |
| Panel de administración    | Ninguno                                  | Web UI con FastAPI + React            | Es un segundo producto. No aporta nada a un asistente que se maneja por voz.                                                            |

## 14. Respuestas directas a las preguntas iniciales

**¿Basta gpt-4o-mini?** Sí, de sobra. La tarea es clasificación de intención con extracción de parámetros. Con la arquitectura local-first, los \$5 alcanzan para ~21,000 comandos, cerca de 2 años de uso normal entre dos personas.

**¿Los \$5 de OpenAI son la única inversión?** Sí. Todo el resto del stack es open source o freeware. El único costo adicional es ~2-3 GB de disco para los modelos. La salvedad no es de dinero sino de funcionalidad: con Spotify Free el control de reproducción va por SMTC y `startfile` en vez de por la API, con anuncios y shuffle ocasional (\$3).

**¿Cómo se divide el trabajo entre dos?** Hemsy toma el eje del audio y la interfaz ( `ears/` , `voice/` , `ui/` ); Rafael toma el eje de la decisión y la acción ( `brain/` , `skills/` ). Se acuerdan dos contratos en la Fase 0 y a partir de ahí trabajan en paralelo sin bloquearse, cada uno contra un doble del otro (§6).